

Приватний заклад вищої освіти
Одеський технологічний університет «ШАГ»
Кафедра інформаційних технологій та фундаментальної підготовки

випускна кваліфікаційна робота бакалавра

**Інформаційна платформа розповсюдження мультимедійного контенту за
моделлю підписки (на прикладі сервісу Bazinga)**

на здобуття бакалаврського рівня вищої освіти
зі спеціальності «F3 Комп'ютерні науки»

Виконавці проекту:

№	П.І.Б.	Ролі	Група
1	Маймескул Єгор Олегович	Backend / Auth	КНП-221
2	Руденко Олена Костянтинівна	Backend / Content	КНП-221

Автор звіту: Руденко Олена Костянтинівна

Керівник	Доцент кафедри ІТ та ФП, к.т.н., [Прізвище І. П. керівника]
----------	---

Одеса – 2025

АНОТАЦІЯ

УДК 004.9

P-83

Руденко О. К. Інформаційна платформа розповсюдження мультимедійного контенту за моделлю підписки (на прикладі сервісу Bazinga): підсистема каталогу контенту, агрегатора метаданих та потокового відео. Пояснювальна записка до дипломного проєкту на здобуття бакалаврського рівня вищої освіти зі спеціальності «F3 Комп'ютерні науки». — Одеса: ОТУШ, 2025. — 40 с.

Об'єкт розробки — серверна та клієнтська складові контентної підсистеми, яка виконує функції агрегування, уніфікованого подання та розповсюдження метаданих мультимедійного контенту (коміксів, манги, аніме та серіалів) з відкритих зовнішніх джерел, а також потокового перегляду відео у складі розподіленої інформаційної системи за моделлю підписки.

Мета роботи — ефективне розроблення відмовостійкого та швидкодіяного програмного інтерфейсу прикладного рівня (REST API) до агрегатора метаданих, який уніфікує різноманітні зовнішні джерела в єдиній моделі контенту й функціонує в умовах обмежень темпу звернень та можливих відмов провайдерів.

Методи дослідження та інструментарій — методи системного аналізу, порівняльного аналізу джерел даних і проєктних рішень та об'єктно-орієнтованого моделювання предметної галузі. Інструментальною базою обрано платформу ASP.NET Core 9 (мова C#) [1] із кешуванням у пам'яті засобами `IMemoryCache` [3] та фабрикою HTTP-клієнтів `IHttpClientFactory`; клієнтський застосунок на React 18 [9] і TypeScript 5 [10] зі складанням засобами Vite 5 [11] та бібліотекою керування серверним станом TanStack Query [12]; відкриті джерела метаданих без авторизації — Jikan v4 (MyAnimeList) [13], akabab/superhero-api [16], TVMaze [14] та Open Library [15].

Результати та їх новизна — синтезовано уніфіковану інформаційну модель контенту та впроваджено комунікаційний протокол на принципах REST. Новизною є не просте відтворення відомого ресурсу, а удосконалений варіант: єдиний інтерфейсний елемент подання різноманітного контенту, стратегія кешування «кеш-обабіч» (Cache-Aside) з диференційованими інтервалами актуальності й обмеженням темпу звернень для обходу лімітів зовнішніх джерел, а також відв'язування клієнта від конкретного постачальника метаданих через комунікаційні інтерфейси, що дозволило замінити припинений комерційний сервіс відкритим джерелом без зміни клієнтської частини.

Основні характеристики та показники — реалізовано єдину точку доступу до метаданих (18 кінцевих точок); статичний набір даних персонажів обсягом близько 1 МБ завантажується одноразово й обробляється у пам'яті; повторні звернення до каталогу обслуговуються з кешу за час порядку мілісекунд.

Інформація щодо впровадження — підсистему розгорнуто у середовищі Docker; обмін із клієнтом здійснюється комунікаційним протоколом REST поверх HTTP; метадані агрегуються з чотирьох відкритих джерел і кешуються у пам'яті серверного процесу.

Взаємозв'язок з іншими роботами — робота є складовою командного проєкту; контентна підсистема взаємодіє з підсистемою облікових записів, профілів та підписок, що виконується іншим учасником. Загальний висновок проєкту є сукупністю звітів усіх учасників.

Сфера застосування — вебсистеми розповсюдження цифрового контенту, агрегатори метаданих; підхід поширюється на суміжні класи систем е-комерції та е-послуг.

Актуальність роботи обумовлена потребою побудови відмовостійких контентних шарів, здатних агрегувати різноманітні зовнішні джерела й зберігати працездатність за їх часткової відмови чи обмежень темпу звернень; результати придатні для повторного використання наступними розробниками як обґрунтований шаблон агрегатора метаданих.

Висновки — спроектовано, реалізовано та випробувано контентну підсистему; усі поставлені у вступі задачі виконано. Перспективою є впровадження адаптивного потокового відео та рекомендаційної підсистеми.

При складанні звіту використано 18 посилань на друковані та електронні джерела інформації.

ЗМІСТ

- АНОТАЦІЯ
 - ВСТУП
 - РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ
 - РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РЕАЛІЗАЦІЯ КОНТЕНТНОЇ ПІДСИСТЕМИ
 - РОЗДІЛ 3. ВИПРОБУВАННЯ ТА ОЦІНКА ПІДСИСТЕМИ
 - ВИСНОВКИ
 - ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ
 - ДОДАТОК А. ТЕХНІЧНЕ ЗАВДАННЯ ПРОЄКТУ
-

ВСТУП

Сучасний стан предметної галузі визначається стрімким зростанням обсягів мультимедійного контенту та підвищенням вимог до якості його подання — наявності обкладинок, описів, жанрів, рейтингів — і швидкості доступу. Сучасні інформаційні системи розповсюдження контенту дедалі частіше будуються як агрегатори, що поєднують контент і метадані з кількох джерел, оскільки побудова власної редакційної бази на десятки тисяч одиниць контенту є ресурсомісткою. Якість такого контентного шару безпосередньо визначає зручність каталогізації, пошуку та споживання контенту.

Світові підходи до розв'язання зазначених завдань передбачають уніфіковане подання різноманітного контенту, наскрізний пошук за кількома категоріями одночасно, персональні колекції на рівні профілю та потоковий перегляд без завантаження файлів. На рівні інженерних рішень типовими є кешування зовнішніх звернень для зниження навантаження й затримок, а також відв'язування системи від конкретного постачальника даних задля стійкості до змін у його доступності. Аналіз провідних систем (детально — у розділі 1) доводить, що жодна з них не поєднує єдиного каталогу різноманітного контенту, наскрізного пошуку та незалежності від конкретного джерела метаданих.

Окремою тенденцією є відмова від побудови власної редакційної бази метаданих на користь агрегування відкритих джерел: це знижує вартість супроводу контенту, проте переносить складність у площину інтеграції — узгодження різних форматів відповідей, дотримання обмежень темпу звернень та забезпечення стійкості до часткової відмови постачальників. Саме ці інженерні виклики, а не наповнення контентом, визначають якість контентного шару й тому стають предметом інженерного дослідження у цій роботі.

Актуальність роботи обумовлена потребою у відмовостійких контентних шарах, що зберігають працездатність за часткової відмови чи обмежень зовнішніх джерел. Побудова такого шару є передумовою функціонування сучасних систем розповсюдження контенту та суміжних систем електронної комерції контенту й електронних послуг, а отриманий шаблон агрегатора метаданих придатний для повторного використання.

Метою роботи є ефективне розроблення відмовостійкого програмного інтерфейсу прикладного рівня (REST API) до кешувального агрегатора метаданих, що уніфікує доступ до різноманітних зовнішніх джерел і забезпечує стабільну роботу підсистеми в умовах обмежень темпу звернень та можливих відмов провайдерів. Уніфіковане візуальне подання карток в інтерфейсі користувача є наслідком цієї уніфікації на рівні моделі та API, а не самостійним предметом дослідження.

Об'єктом дослідження виступає процес проєктування контентної підсистеми розподіленої інформаційної системи — агрегатора метаданих, уніфікованого каталогу, наскрізного пошуку, персональних колекцій та потокового відео. Предметом дослідження є архітектурні підходи, шаблони проєктування та технології кешування, які забезпечують відмовостійкість, швидкодію й підмінність джерел даних. Завдання вирішуються в умовах неповної визначеності: зовнішні джерела мають різні формати відповідей, накладають обмеження на темп звернень, можуть бути тимчасово недоступними, а окремі з них — узагалі припинятися. Зокрема, у ході роботи комерційний сервіс метаданих коміксів став недоступним, що потребувало дослідницького переходу на альтернативне відкрите джерело без зміни клієнтської частини, — це підтверджує недетермінований характер задачі.

Для досягнення поставленої мети сформульовано такі завдання дослідження:

1. провести порівняльне дослідження наявних систем розповсюдження контенту та доступних зовнішніх джерел метаданих за програмними інтерфейсами доступу, встановити критерії вибору джерел;
2. синтезувати уніфіковану інформаційну модель контенту та спроектувати кешувальний агрегатор за стратегією «кеш-обабіч» із диференційованими інтервалами актуальності;
3. реалізувати серверні комунікаційні інтерфейси та сервіси інтеграції з джерелами метаданих за принципом інверсії залежностей; впровадити обмеження темпу звернень для дотримання лімітів зовнішніх джерел;
4. забезпечити підмінність постачальника метаданих через комунікаційний інтерфейс і реалізувати уніфікований каталог, наскрізний пошук та потоковий перегляд відео;
5. реалізувати підсистему персональних колекцій з прив'язкою до профілю користувача;
6. провести випробування підсистеми, зокрема в умовах відмови та обмежень зовнішніх джерел, та оцінити відповідність технічному завданню.

Методи дослідження базуються на поєднанні методів системного аналізу, порівняльного аналізу аналогів та об'єктно-орієнтованого моделювання предметної галузі.

Взаємозв'язок з іншими складовими проєкту. Ця робота є складовою командного проєкту: цей звіт зосереджено на контентній підсистемі, яка взаємодіє з підсистемою облікових записів, профілів та підписок, що розробляється іншим учасником команди. Початкові розділи, які описують загальну характеристику системи, є спільними для всієї команди; починаючи з розділу, присвяченого контентній підсистемі, матеріал викладено виключно як авторський.

РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ

1.1 Опис предметної галузі

Інформаційна система, що проектується (робоча назва — Bazinga), є розподіленою клієнт-серверною системою розповсюдження мультимедійного контенту за моделлю підписки. Система поєднує дві предметні складові у межах єдиного облікового запису: *Bazinga Comics* — каталог коміксів, манги та персонажів із читанням випусків онлайн, та *BazingaTV* — потоковий перегляд відеоконтенту (аніме, мультсеріали, супергеройські серіали).

Контентна підсистема, якій присвячено цей звіт, реалізує єдину точку доступу до метаданих, уніфікований каталог, наскрізний пошук, потоковий перегляд та персональні колекції. Її ключовою властивістю є незалежність від конкретного джерела даних: контент надходить із відкритих метаджерел, проте структура системи побудована так, щоб працювати з будь-якою контентною базою без зміни клієнтської частини. Основним фокусом роботи є не ручне наповнення даних, а побудова універсального контентного шару — агрегатора метаданих, системи кешування, пошуку та єдиного інтерфейсу для всіх типів контенту.

Архітектурно систему побудовано за стилем «односторінковий застосунок (SPA) поверх комунікаційного протоколу REST». Серверна частина застосовує кешування у пам'яті [3] та фабрику HTTP-клієнтів для звернень до зовнішніх джерел; клієнтська частина уніфікує подання різноманітного контенту спільними інтерфейсними компонентами на основі React [9] і TypeScript [10].

1.2 Порівняльний аналіз наявних рішень

Для обґрунтування вибору взірцевої системи та визначення напрямів удосконалення виконано порівняльний аналіз найвідоміших систем розповсюдження контенту за ознаками, що стосуються виявлення та споживання контенту (табл. 1.1). Як значення ознаки використано «+» (наявна), «-» (відсутня) або «частково» (наявна у спрощеному вигляді).

Таблиця 1.1 — Порівняння систем розповсюдження контенту за засобами виявлення та споживання

Ознака / Система	Netflix	Crunchyroll	ComiXology	Bazinga
Єдиний каталог відео, коміксів і манги	—	—	—	+
Наскрізний пошук одночасно за кількома типами контенту	частково	—	—	+
Уніфіковане подання різноманітного контенту	—	—	—	+
Персональні колекції на рівні профілю	+	частково	—	+

Ознака / Система	Netflix	Crunchyroll	ComiXology	Bazinga
Незалежність від конкретного джерела метаданих	–	–	–	+
Відмовостійке агрегування з кешуванням	частково	–	–	+

Як свідчить таблиця 1.1, жоден рядок не містить ознаки одночасно в усіх системах, що підтверджує диференційність обраних критеріїв. Серед аналогів персональні колекції на рівні профілю найповніше реалізовано в Netflix, який і обрано за взірцем щодо цієї властивості. Водночас жодна з наявних систем не поєднує єдиного каталогу різноманітного контенту з наскрізним пошуком, уніфікованим поданням та незалежністю від конкретного джерела метаданих — саме усунення цих «мінусів» становить предмет удосконалення.

1.3 Постановка задачі розроблення

Проведений аналіз засвідчив, що готові продукти орієнтовані на один тип контенту, не передбачають підмінності джерел даних та не поєднують уніфікованого подання з наскрізним пошуком. З огляду на це, розроблення власної контентної підсистеми є доцільним: воно дозволяє усунути виявлені вади шляхом побудови агрегатора метаданих з уніфікованою моделлю контенту, стратегією кешування й незалежністю від конкретного постачальника.

До функціональних вимог підсистеми належать: єдина точка доступу до метаданих з кількох зовнішніх джерел; уніфіковане подання різноманітного контенту; фільтрація за жанрами/видавцями та посторінкове гортання; виключення контенту для дорослих із загальнодоступних добірок; наскрізний пошук одночасно за коміксами, мангою, аніме та персонажами з відкриттям деталей у накладенні; персональні колекції з прив'язкою до профілю; потоковий перегляд відео у вбудованому програвачі. До нефункціональних вимог належать: мінімізація кількості звернень до зовнішніх джерел кешуванням; дотримання обмежень темпу звернень; обслуговування повторних звернень з кешу за час порядку мілісекунд; збереження працездатності за відмови зовнішнього джерела; адаптивність інтерфейсу. Повний текст технічного завдання наведено у Додатку А.

Висновок до розділу. У межах роботи реалізується контентна підсистема удосконаленої системи розповсюдження контенту, яка усуває виявлені недоліки наявних аналогів. Використання готового стороннього рішення є недоцільним через орієнтацію аналогів на один тип контенту та відсутність підмінності джерел. Це обґрунтовує предмет і задачі подальшого проєктування, викладеного у розділі 2.

РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РЕАЛІЗАЦІЯ КОНТЕНТНОЇ ПІДСИСТЕМИ

2.1 Обґрунтування вибору технологій та джерел метаданих

Вибір технологій виконувався дослідницьким методом: для кожного компонента опрацьовувалися альтернативи й обирався варіант, що найкраще відповідає критеріям швидкодії, відмовостійкості та простоти супроводу. Серверну частину реалізовано на платформі ASP.NET Core 9 [1] з кешуванням у пам'яті засобами `IMemoryCache` [3] та фабрикою типізованих HTTP-клієнтів `IHttpClientFactory`. Клієнтську частину реалізовано мовою TypeScript [10] на основі React [9] зі складанням засобами Vite [11]; керування серверним станом виконує бібліотека TanStack Query [12]. Потоковий перегляд відео реалізовано вбудованим програвачем стандарту HTML5 [8] без сторонніх залежностей.

Окремою дослідницькою задачею став вибір джерел метаданих. Більшість галузевих API (Marvel, ComicVine, TMDB) вимагають реєстрації, ключів доступу та підпису кожного звернення, а окремі з них не надають обкладинок. Тому за критеріями відсутності обов'язкової авторизації, наявності обкладинок, описів і жанрів та стабільності доступу проведено порівняльний аналіз (табл. 2.1). Як значення ознаки використано «+», «—» або «частково».

Таблиця 2.1 — Порівняльний аналіз джерел метаданих

Джерело	Без авторизації	Обкладинки	Описи	Жанри	Стабільність доступу
Marvel Developer API	—	+	+	+	—
ComicVine	—	+	+	+	+
TMDB	—	+	+	+	+
Jikan v4 (MyAnimeList)	+	+	+	+	+
akabab/superhero-api	+	+	+	частково	+
TVMaze	+	+	+	+	+
Open Library	+	+	+	частково	+

За результатами аналізу для аніме й манги обрано Jikan v4 [13], для персонажів коміксів — akabab/superhero-api [16], для серіалів — TVMaze [14], для коміксів — Open Library [15]. Джерела, що потребують авторизації або підпису кожного звернення,

відхилено; додатковим підтвердженням ризику залежності від комерційного джерела стало припинення доступу до Marvel Developer API під час виконання роботи, після чого джерело коміксів було замінено на Open Library без зміни клієнтської частини (детально — п. 2.7).

2.2 Архітектура підсистеми та принципи проєктування

Контентну підсистему спроектовано за тришаровою організацією з чітким поділом відповідальностей: рівень подання (контролер метаданих), рівень бізнес-логіки (сервіси інтеграції з джерелами) та рівень доступу до даних (зовнішні джерела й кеш у пам'яті). Проектні рішення підпорядковано принципам SOLID. Принцип єдиної відповідальності реалізовано закріпленням за кожним сервісом однієї групи джерел. Принцип відкритості/закритості забезпечено тим, що для підключення нового джерела метаданих достатньо додати новий сервіс через інверсію залежностей, не змінюючи наявного коду. Принцип інверсії залежностей реалізовано тим, що контролер залежить від інтерфейсів сервісів (`IJikanService`, `ISuperheroService`, `IComicMetadataService`), а не від конкретних реалізацій.

Серед шаблонів проєктування застосовано: `Cache-Aside` — для кешування відповідей зовнішніх джерел; `Factory` — через фабрику HTTP-клієнтів `IHttpClientFactory`; `DTO` — для уніфікованого подання різноманітного контенту у відповідях; узагальнений (generic) метод — як форму повторного використання логіки кешованого гортання. Така організація знижує зв'язність складових, спрощує супровід та забезпечує підмінність джерел даних.

Застосування фабрики HTTP-клієнтів замість безпосереднього створення екземплярів `HttpClient` має практичне обґрунтування: воно усуває відому проблему вичерпання сокетів при частому створенні клієнтів, дозволяє централізовано налаштовувати базову адресу, тайм-аути й заголовки для кожного джерела, а також спрощує підміну реалізації під час модульного тестування. Кешування у пам'яті процесу обрано замість зовнішнього кеш-сховища з огляду на характер даних (метадані змінюються повільно) та на прагнення мінімізувати кількість інфраструктурних залежностей на етапі прототипу; за потреби масштабування реалізацію кешу можна замінити розподіленою без зміни прикладної логіки, оскільки доступ до кешу інкапсульовано.

2.3 Агрегатор метаданих та єдина точка доступу

Єдину точку доступу до метаданих реалізовано контролером `MetadataController`, який об'єднує різноманітні джерела через сервіси та надає клієнту уніфіковані відповіді. Завдяки цьому клієнт працює лише з уніфікованою моделлю даних і не залежить від конкретних провайдерів, що дозволяє у майбутньому змінити або замінити джерело без

зміни клієнтської частини. Контролер містить 18 кінцевих точок за всіма категоріями контенту — аніме, манга, персонажі, серіали та комікси — і залежить від інтерфейсів сервісів (Приклад 1).

Приклад 1 — Єдина точка доступу до метаданих з інверсією залежностей (Controllers/MetadataController.cs)

```
public MetadataController(IJikanService jikan, ISuperheroService supers,
    IComicMetadataService comics)
{
    _jikan = jikan; _supers = supers; _comics = comics;    // залежність від інтерфейсів
}

[HttpGet("comics")]
public Task<ComicsMetaPagedResponse> Comics(
    [FromQuery] int page = 1, [FromQuery] int limit = 24, [FromQuery] string? q = null,
    CancellationToken ct = default)
    => _comics.ListAsync(Math.Max(1, page), Math.Clamp(limit, 1, MaxComicsLimit), q, ct);

[HttpGet("anime/top")]
public Task<JikanPagedResponse<AnimeDto>> TopAnime(
    [FromQuery] int page = 1, [FromQuery] int limit = 12, CancellationToken ct = default)
    => _jikan.TopAnimeAsync(Math.Max(1, page), Clamp(limit), ct);
```

Логіку інтеграції винесено у сервіси за інтерфейсами відповідно до принципу інверсії залежностей: `JikanService` обслуговує аніме й мангу, `SuperheroService` — персонажів і серіали, `OpenLibraryComicService` — комікси. Клієнт не знає, яке джерело стоїть за кінцевою точкою, тому в майбутньому, при переході на ліцензовану базу даних, достатньо замінити реалізацію відповідного інтерфейсу — контролер і клієнтська частина залишаються незмінними.

2.4 Кешування за стратегією Cache-Aside

Оскільки зовнішні джерела мають обмеження темпу звернень, а окремі з них повертають великі обсяги даних, для зниження кількості звернень і затримок застосовано стратегію кешування «кеш-обабіч» (Cache-Aside) [6] над кешем у пам'яті. Інтервали актуальності диференційовано за характером даних: списки — короткий інтервал (15 хвилин), деталі — середній, статичні набори — тривалий (12 годин). Для уникнення дублювання коду між десятком майже однакових методів отримання даних реалізовано узагальнений метод кешованого посторінкового отримання, параметризований сирим типом постачальника та власним типом моделі (Приклад 2). Без нього довелося б підтримувати десять майже ідентичних копій; натомість усі зміни логіки кешування зосереджено в одному місці.

Приклад 2 — Узагальнений метод кешованого посторінкового отримання даних за стратегією Cache-Aside (Services/JikanService.cs)

```
private async Task<JikanPagedResponse<TDto>> GetPagedCached<TRaw, TDto>(
    string path, TimeSpan ttl, Func<TRaw, TDto> map, CancellationToken ct)
    where TRaw : class
{
    return (await _cache.GetOrCreateAsync($"path:{path}", async entry =>
```

```

{
    entry.AbsoluteExpirationRelativeToNow = ttl;
    var raw = await GetJsonAsync<JikanEnvelope<List<TRaw>>>(path, ct);
    return new JikanPagedResponse<TDto>
    {
        Data = raw?.Data?.Select(map).ToList() ?? new(),
        Pagination = new JikanPagination
        {
            CurrentPage = raw?.Pagination?.CurrentPage ?? 1,
            LastVisiblePage = raw?.Pagination?.LastVisiblePage ?? 1,
            HasNextPage = raw?.Pagination?.HasNextPage ?? false,
            TotalItems = raw?.Pagination?.Items?.Total,
        },
    };
    }));
}

```

Завдяки кешуванню повторні переходи каталогом не призводять до звернень у мережу, а обмеження темпу зовнішнього джерела не вичерпується навіть під час інтенсивного гортання. Інтервал актуальності для списків задано константою (15 хвилин), що є компромісом між свіжістю даних та кількістю зовнішніх звернень.

2.5 Обмеження темпу звернень та фільтрація контенту

Зовнішнє джерело Jikan накладає обмеження темпу звернень — близько трьох на секунду. Під час одночасного формування кількох добірок (на головній сторінці BazingaTV) частина звернень відхилялася зі статусом «занадто багато запитів», унаслідок чого окремі ряди залишалися порожніми. Для усунення цього синтезовано механізм обмеження темпу звернень: усі звернення серіалізуються процесним семафором із мінімальним інтервалом між ними та одним повтором після паузи у разі відхилення (Приклад 3). Це гарантує дотримання обмежень джерела без втрати даних.

Приклад 3 — Обмеження темпу звернень до зовнішнього джерела з одним повтором (Services/JikanService.cs)

```

// Серіалізуємо звернення процесним семафором із мінімальним інтервалом, щоб
// не перевищувати ~3 запити/с і не отримувати відмов «занадто багато запитів».
private static readonly SemaphoreSlim Gate = new(1, 1);
private static readonly TimeSpan MinSpacing = TimeSpan.FromMilliseconds(380);
private static DateTime _lastCallUtc = DateTime.MinValue;

private async Task<HttpResponseMessage> SendThrottledAsync(string path, CancellationToken ct)
{
    await Gate.WaitAsync(ct);
    try
    {
        {
            var since = DateTime.UtcNow - _lastCallUtc;
            if (since < MinSpacing) await Task.Delay(MinSpacing - since, ct);
            try { return await _http.GetAsync(path, ct); }
            finally { _lastCallUtc = DateTime.UtcNow; }
        }
    }
    finally { Gate.Release(); }
}

private async Task<T?> GetJsonAsync<T>(string path, CancellationToken ct)
{
    for (var attempt = 0; attempt < 2; attempt++)
    {

```

```

        using var resp = await SendThrottledAsync(path, ct);
        if (resp.StatusCode == System.Net.HttpStatusCode.TooManyRequests)
        {
            if (attempt == 0) { await Task.Delay(TimeSpan.FromMilliseconds(900), ct);
continue; }
            return default;
        }
        if (!resp.IsSuccessStatusCode) return default;
        return await resp.Content.ReadFromJsonAsync<T>(Json, ct);
    }
    return default;
}

```

Для виключення контенту для дорослих із загальнодоступних добірок реалізовано фільтр жанрів манги, який доповнює недостатній штатний параметр джерела: штатний параметр виключає лише одну категорію, тому додатково задано явний перелік виключених жанрів (Приклад 4).

Приклад 4 — Виключення жанрів для дорослих із добірок манги (`Services/JikanService.cs`)

```

// 9 = Ecchi, 12 = Hentai, 28 = Boys Love, 30 = Girls Love, 49 = Erotica.
// Штатний параметр sfw=true фільтрує лише Hentai, тож доповнюємо явним списком.
private const string MangaExcludedGenres = "9,12,28,30,49";

public Task<JikanPagedResponse<MangaDto>> SearchMangaAsync(
    string? query, int page, int limit, int? genre, string? status, string? orderBy,
    CancellationToken ct)
{
    var url = BuildUrl("manga", q =>
    {
        q["page"] = page.ToString(); q["limit"] = limit.ToString();
        if (!string.IsNullOrEmpty(query)) q["q"] = query;
        q["sfw"] = "true";
        q["genres_exclude"] = MangaExcludedGenres;
    });
    return GetPagedCached<JikanManga, MangaDto>(url, ListCacheTtl, MapManga, ct);
}

```

2.6 Bulk-обробка статичного набору даних персонажів

Джерело персонажів `akabab/superhero-api` [16] повертає один великий статичний набір даних обсягом близько 1 МБ (приблизно 731 запис). Замість посторінкових звернень застосовано bulk-стратегію: увесь набір завантажується одним зверненням і кешується на тривалий інтервал (12 годин), після чого пошук, фільтрація та посторінкове гортання виконуються повністю у пам'яті (Приклад 5). Це усуває мережеві звернення під час взаємодії з каталогом і робить роботу сторінки незалежною від поточної доступності джерела.

Приклад 5 — Фільтрація та посторінкове гортання статичного набору у пам'яті (`Services/SuperheroService.cs`)

```

public async Task<JikanPagedResponse<SuperheroDto>> ListAsync(
    int page, int limit, string? query, string? publisher, string? alignment,
    CancellationToken ct)
{
    var all = await LoadAllAsync(ct); // одноразове завантаження + кеш на 12

```

```

    год
    IEnumerable<SuperheroDto> filtered = all;
    if (!string.IsNullOrEmpty(publisher))
        filtered = filtered.Where(h => string.Equals(h.Publisher, publisher,
StringComparison.OrdinalIgnoreCase));

    var list = filtered.ToList();
    var safePage = Math.Max(1, page);
    var safeLimit = Math.Clamp(limit, 1, 48);
    var pageItems = list.Skip((safePage - 1) * safeLimit).Take(safeLimit).ToList();
    return new JikanPagedResponse<SuperheroDto>
    {
        Data = pageItems,
        Pagination = new JikanPagination
        {
            CurrentPage = safePage,
            LastVisiblePage = Math.Max(1, (int)Math.Ceiling(list.Count /
(double)safeLimit)),
            HasNextPage = safePage * safeLimit < list.Count,
            TotalItems = list.Count,
        },
    };
}

```

Завдяки bulk-стратегії перемикавання фільтра видавців (наприклад, Marvel → DC → Image) відбувається миттєво, без жодного мережевого звернення, оскільки фільтрація — це звичайний запит мовою LINQ по списку в пам'яті.

2.7 Підмінність джерела: інтеграція Open Library

Однією з умов невизначеності стало припинення доступу до комерційного сервісу метаданих коміксів під час виконання роботи. Завдяки відв'язуванню клієнта від постачальника через інтерфейс `IComicMetadataService` джерело коміксів було замінено відкритим сервісом Open Library [15] без зміни ані контролера, ані клієнтської частини. Базову добірку сформовано як об'єднання керованих запитів за супергеройськими франшизами з усуненням повторів та чергуванням джерел (Приклад 6); вільнотекстовий пошук обмежено предметною рубрикою коміксів. Сирі відповіді відображаються в уніфіковану модель: побудова адрес обкладинок, виокремлення авторів і року (Приклад 7).

Приклад 6 — Кероване об'єднання запитів з усуненням повторів і чергуванням франшиз (`Services/OpenLibraryComicService.cs`)

```

private async Task<List<ComicMetaDto>> LoadCuratedAsync(Cancellation token ct)
{
    return (await _cache.GetOrCreateAsync("comics:ol:curated", async entry =>
    {
        entry.AbsoluteExpirationRelativeToNow = CuratedTtl;
        var tasks = CuratedFranchises.Select(f => SearchRawAsync(f, 1, 18, ct));
        var results = await Task.WhenAll(tasks);

        // Чергуємо франшизи (round-robin), усуваємо повтори за id, вимагаємо обкладинку.
        var byFranchise = results.Select(r => r.Data).ToList();
        var seen = new HashSet<int>();
        var merged = new List<ComicMetaDto>();
        var max = byFranchise.Count == 0 ? 0 : byFranchise.Max(l => l.Count);
        for (var i = 0; i < max; i++)
            foreach (var list in byFranchise)

```

```

        {
            if (i >= list.Count) continue;
            var item = list[i];
            if (item.Image is null) continue;
            if (seen.Add(item.Id)) merged.Add(item);
        }
        return merged;
    }
}

```

Приклад 7 — Відображення сирової відповіді у уніфіковану модель та побудова адрес обкладинок (`Services/OpenLibraryComicService.cs`)

```

private static ComicMetaDto MapDoc(OLDoc d) => new()
{
    Id = ParseWorkId(d.Key), // "/works/OL123W" → 123
    Title = d.Title ?? "Untitled",
    Image = CoverUrl(d.CoverId, "L"),
    Thumbnail = CoverUrl(d.CoverId, "M"),
    Year = d.FirstPublishYear,
    Creators = (d.AuthorName ?? new()).Take(4).ToList(),
    Genres = CleanSubjects(d.Subject),
};

private static string? CoverUrl(int? coverId, string size) =>
    coverId is > 0 ? $"https://covers.openlibrary.org/b/id/{coverId}-{size}.jpg" : null;

```

Цей сценарій підтверджує цінність відв'язування від постачальника: підміна джерела торкнулася лише однієї реалізації інтерфейсу, тоді як решта системи лишилася незмінною.

2.8 Уніфікований інтерфейс контенту

Основною архітектурною ідеєю клієнта є єдиний компонент картки для всіх типів контенту. Завдяки цьому комікси, манга, аніме та серіали відображаються однаково, а нові типи контенту інтегруються без дублювання коду. За відмови завантаження обкладинки застосовується градієнтний заміник із назвою, що зберігає цілісність сітки (Приклад 8). Горизонтальні ряди реалізовано у стилі Netflix зі стрілками гортання та прихованим скролбаром; ряд може містити одночасно різні типи контенту, оскільки тип зберігається в обробнику кліку.

Приклад 8 — Уніфікована картка з градієнтним заміником при відмові завантаження обкладинки (`components/MediaCard.tsx`)

```

<div className="relative aspect-[2/3] overflow-hidden">
  <div aria-hidden className={cn("absolute inset-0 bg-gradient-to-br", fallbackBg)} />
  <div aria-hidden className="absolute inset-0 grid place-items-center text-center px-3">
    <span className="text-xs font-black uppercase tracking-wider text-white/85 line-clamp-3">
      {title}
    </span>
  </div>
  <img
    src={resolveImageUrl(image)} alt={title} loading="lazy"
    className="relative h-full w-full object-cover transition-transform duration-500 group-hover:scale-110"
    onError={(e) => ((e.currentTarget as HTMLImageElement).style.opacity = "0")}
  />

```

```
    />  
  </div>
```

2.9 Наскрізний пошук та потокове відео

Наскрізний пошук виконує паралельні звернення за кількома типами контенту одночасно (комікси, персонажі, манга, аніме) з усуненням «миготіння» введення (debounce) і відкриває детальне подання результату безпосередньо в накладенні, без переходу на іншу сторінку, завдяки чому користувач не втрачає контекст. Для коректного позиціонування накладення винесено у портал кореневого вузла документа, що усуває його обрізання батьківськими елементами (Приклад 9).

Приклад 9 — Наскрізний пошук: паралельні звернення та винесення накладення у портал (components/MasterSearch.tsx)

```
const heroes = useQuery({ queryKey: ["ms:characters", debounced],  
  queryFn: () => fetchSuperheroes({ q: debounced, limit: 5 }), enabled, staleTime:  
60_000 });  
const manga = useQuery({ queryKey: ["ms:manga", debounced],  
  queryFn: () => searchManga({ q: debounced, limit: 5 }), enabled, staleTime: 60_000 });  
const anime = useQuery({ queryKey: ["ms:anime", debounced],  
  queryFn: () => searchAnime({ q: debounced, limit: 5 }), enabled, staleTime: 60_000 });  
const metaIssues = useQuery({ queryKey: ["ms:comics", debounced],  
  queryFn: () => fetchComicsMeta({ q: debounced, limit: 5 }), enabled, staleTime: 60_000 });  
  
// Накладення виноситься у портал кореня документа, щоб його не «обрізав» хедер.  
return createPortal(overlay, document.body);
```

Потоковий перегляд відео реалізовано вбудованим програвачем стандарту HTML5 [8] без сторонніх SDK, завдяки чому розмір клієнтського пакета не зростає за рахунок зовнішнього програвача. Героїчний блок головної сторінки BazingaTV програвє трейлери випадковими фрагментами тривалістю 10–15 секунд, тому кожне відвідування показує інший фрагмент. Сценарій «знайти → відкрити → переглянути» повністю реалізовано в межах односторінкового застосунку.

2.10 Персональні колекції на рівні профілю

Персональні колекції реалізовано окремою структурною сутністю, відв'язаною від таблиці локальних коміксів, що дозволяє зберігати одиниці контенту з будь-якого зовнішнього джерела. Колекції прив'язано до профілю: активний профіль визначається із заголовка звернення та звіряється з власником облікового запису (Приклад 10). Дві колекції — «Бібліотека» (комікси й манга) та «Мій список» (аніме й серіали) — розрізняються дискримінатором, що зменшує кількість структурних сутностей. На клієнті контекст колекцій перезавантажується при зміні активного профілю, що й забезпечує окремі колекції для кожного профілю (Приклад 11).

Приклад 10 — Додавання до колекції з прив'язкою до профілю та звіркою власника (Controllers/CollectionsController.cs)

```
[HttpPost("{collection}")]
public async Task<ActionResult<IEnumerable<CollectionItemDto>>> Add(
    string collection, [FromBody] CollectionItemRequest req, CancellationToken ct)
{
    collection = collection.ToLowerInvariant();
    if (!AllowedCollections.Contains(collection)) return NotFound();
    var profile = await ResolveProfileAsync(ct); // профіль із заголовка, звірений із
    власником
    if (profile is null) return Unauthorized();

    var exists = await _db.CollectionItems.AnyAsync(
        i => i.ProfileId == profile.Id && i.Collection == collection
        && i.Kind == req.Kind && i.ContentId == req.ContentId, ct);
    if (!exists)
    {
        _db.CollectionItems.Add(new ProfileCollectionItem {
            ProfileId = profile.Id, Collection = collection, Kind =
            req.Kind.ToLowerInvariant(),
            ContentId = req.ContentId.Trim(), Title = req.Title, Image = req.Image,
            Subtitle = req.Subtitle, PayloadJson = req.PayloadJson, AddedAt =
            DateTime.UtcNow });
        try { await _db.SaveChangesAsync(ct); }
        catch (DbUpdateException) { /* гонитва за унікальним індексом – ігноруємо */ }
    }
    return Ok(await LoadItemsAsync(profile.Id, collection, null, ct));
}
```

Приклад 11 — Перезавантаження персональних колекцій при зміні активного профілю (`contexts/CollectionsContext.tsx`)

```
const reload = useCallback(async () => {
    if (!token || profileId == null) { setLibrary([]); setMyList([]); return; }
    const [lib, list] = await Promise.all([
        listCollection(token, profileId, "library"),
        listCollection(token, profileId, "mylist"),
    ]);
    setLibrary(lib); setMyList(list);
}, [token, profileId]);

// Перезавантаження при зміні акаунта чи активного профілю – саме це й забезпечує
// окремі колекції для кожного профілю.
useEffect(() => { void reload(); }, [reload]);
```

2.11 Допоміжні компоненти та реєстрація залежностей

Bulk-стратегію для статичного набору персонажів забезпечує метод одноразового завантаження з кешуванням: набір тягнеться одним зверненням, після чого зберігається у кеші на тривалий інтервал; за відмови джерела повертається порожній перелік із журналюванням помилки, тому сторінка не аварійно завершується (Приклад 12). Це реалізація градаційного зниження функціональності на рівні джерела даних.

Приклад 12 — Одноразове завантаження статичного набору з кешуванням і резервним поверненням (`Services/SuperheroService.cs`)

```
private Task<List<SuperheroDto>> LoadAllAsync(CancellationToken ct) =>
    _cache.GetOrCreateAsync("superheroes:all", async entry =>
    {
        entry.AbsoluteExpirationRelativeToNow = CharactersTtl; // 12 годин
        try
        {
```

```

        using var resp = await _http.GetAsync(AkababAllUrl, ct);
        resp.EnsureSuccessStatusCode();
        var raw = await resp.Content.ReadFromJsonAsync<List<AkababHero>>(JsonOptions,
ct)
            ?? new List<AkababHero>();
        return raw.Select(MapHero).ToList();
    }
    catch (Exception ex)
    {
        _logger.LogWarning(ex, "akabab недоступний – повертаємо порожній перелік");
        return new List<SuperheroDto>(); // graceful degradation
    }
}!!;

```

Перелік жанрів, отриманий від Open Library, містить службові позначки (наприклад, рівні читання чи технічні мітки), тому його очищують перед поданням користувачеві: відкидаються надто довгі рядки, технічні позначки та дублікати, а кількість обмежується (Приклад 13). Це підвищує якість подання без втручання у джерело.

Приклад 13 — Очищення переліку жанрів від службових позначок (Services/OpenLibraryComicService.cs)

```

private static List<string> CleanSubjects(IEnumerable<string>? subjects) =>
    (subjects ?? Enumerable.Empty<string>())
        .Where(s => !string.IsNullOrEmpty(s)
            && !s.Contains(':') && !s.Contains('=') && s.Length <= 28
            && !s.StartsWith("Reading Level", StringComparison.OrdinalIgnoreCase)
            && !s.StartsWith("Accessible book", StringComparison.OrdinalIgnoreCase))
        .Distinct(StringComparer.OrdinalIgnoreCase)
        .Take(6)
        .ToList();

```

Горизонтальні ряди каталогу підтримують як однорядкове, так і дворядкове подання залежно від типу добірки. Перемикання реалізовано вибором класів розмітки: для дворядкового режиму застосовано двовимірну сітку з горизонтальним заповненням, для однорядкового — горизонтальний потік із прив'язкою прокручування (Приклад 14).

Приклад 14 — Перемикання між одно- та дворядковим поданням ряду (components/Rail.tsx)

```

<div
  id={trackId(id)}
  className={cn(
    "overflow-x-auto px-4 md:px-8 pb-2 pt-6 scroll-smooth no-scrollbar",
    rows === 2
      ? "grid grid-rows-2 grid-flow-col auto-cols-max gap-x-4 gap-y-5 md:gap-x-5"
      : "flex gap-4 md:gap-5 snap-x snap-mandatory"
  )}
  style={{ scrollbarWidth: "none" }}
>
  {children}
</div>

```

Сервіси інтеграції з джерелами реєструються у контейнері інверсії залежностей як типізовані HTTP-клієнти з єдиним налаштуванням базової адреси, тайм-ауту та заголовків (Приклад 15). Це є реалізацією шаблону Factory через фабрику HTTP-клієнтів і забезпечує підмінність реалізацій моками під час тестування.

Приклад 15 — Реєстрація сервісів джерел як типізованих HTTP-клієнтів (Program.cs)

```
builder.Services.AddMemoryCache();
builder.Services.AddHttpClient<IJikanService, JikanService>(client =>
{
    client.BaseAddress = new Uri("https://api.jikan.moe/v4/");
    client.Timeout = TimeSpan.FromSeconds(15);
});
builder.Services.AddHttpClient<IComicMetadataService, OpenLibraryComicService>(client =>
{
    client.BaseAddress = new Uri("https://openlibrary.org/");
    client.Timeout = TimeSpan.FromSeconds(15);
});
```

На клієнті звернення до персональних колекцій інкапсульовано у тонкому шарі функцій, що передає ідентифікатор активного профілю у заголовок звернення; це забезпечує прив'язку колекції до профілю на рівні комунікаційного протоколу (Приклад 16).

Приклад 16 — Клієнтський шар звернень до персональних колекцій із передаванням профілю (lib/collections.ts)

```
const profileHeader = (profileId: number | null): Record<string, string> =>
    profileId != null ? { "X-Profile-Id": String(profileId) } : {};

export const addToCollection = (
    token: string, profileId: number | null, collection: CollectionName, input:
    CollectionItemInput) =>
    apiFetch<CollectionItem[]>(`/api/collections/${collection}`, {
        method: "POST", authToken: token, headers: profileHeader(profileId),
        body: JSON.stringify({
            kind: input.kind, contentId: input.contentId, title: input.title,
            image: input.image ?? null, payloadJson: input.payload != null ?
            JSON.stringify(input.payload) : null,
        }),
    });
```

Окремою задачею уніфікації стало приведення різнорідних полів зовнішніх джерел до єдиної моделі. Зокрема, джерело Jikan повертає жанри й теми окремими переліками, тоді як у каталозі вони мають подаватися єдиним списком без повторів. Тому під час відображення сирової сутності у власну модель переліки жанрів і тем об'єднуються, очищуються від порожніх значень та дедуплікуються (Приклад 17). Аналогічно уніфікуються адреси обкладинок (велика та середня) і назва, що дозволяє клієнту працювати з єдиною структурою незалежно від джерела.

Приклад 17 — Відображення сирової сутності аніме у власну модель з об'єднанням жанрів і тем (Services/JikanService.cs)

```
private static AnimeDto MapAnime(JikanAnime a) => new()
{
    MalId = a.MalId,
    Title = a.TitleEnglish ?? a.Title ?? "Untitled",
    Synopsis = a.Synopsis,
    Type = a.Type, Episodes = a.Episodes, Status = a.Status,
    ImageUrl = a.Images?.Jpg?.ImageUrl,
    LargeImageUrl = a.Images?.Jpg?.LargeImageUrl ?? a.Images?.Jpg?.ImageUrl,
    Score = a.Score, Rank = a.Rank, Year = a.Year, Rating = a.Rating,
```

```
// Жанри та теми Jikan об'єднуються в єдиний список без повторів.  
Genres = (a.Genres ?? new()).Concat(a.Themes ?? new())  
    .Select(g => g.Name ?? "").Where(n => !string.IsNullOrEmpty(n)).Distinct().ToList(),  
Studios = (a.Studios ?? new())  
    .Select(s => s.Name ?? "").Where(n => !string.IsNullOrEmpty(n)).ToList(),  
};
```

РОЗДІЛ 3. ВИПРОБУВАННЯ ТА ОЦІНКА ПІДСИСТЕМИ

3.1 Методика та види тестування

Перевірку коректності контентної підсистеми організовано на кількох рівнях відповідно до видів тестування, передбачених технічним завданням. На рівні функціональних сценаріїв застосовано проходження повних користувацьких шляхів у браузерях на базі рушіїв Chromium та Gecko. На рівні відмовостійкості проведено випробування у штучних умовах недоступності та обмежень темпу звернень зовнішніх джерел. На рівні продуктивності виконано вимірювання часу обслуговування повторних звернень із кешу та перевірку дотримання обмежень темпу звернень. Передбачено також інтеграційне, навантажувальне та адаптивне тестування системи у цілому; у межах контентної підсистеми основну увагу приділено функціональному випробуванню кінцевих точок і перевірці стратегій кешування та відмовостійкості.

Методику побудовано за принципом відтворюваності: кожен сценарій формулюється як послідовність кроків із фіксованою очікуваною реакцією системи. Особливу увагу приділено межовим випадкам — одночасному формуванню кількох добірок (що раніше спричиняло порожні ряди через перевищення темпу звернень), недоступності окремого джерела та відсутності обкладинки в окремих одиниць контенту.

3.2 Функціональне випробування

Функціональним випробуванням підтверджено реалізацію всіх вимог технічного завдання: єдиної точки доступу до метаданих; уніфікованого каталогу з фільтрацією та посторінковим гортанням; наскрізного пошуку за кількома типами контенту з відкриттям деталей у накладенні; персональних колекцій на рівні профілю; потокового перегляду відео. Окремо перевірено наповнення всіх рядів каталогу реальними метаданими (обкладинки, описи, жанри), коректне розділення персональних колекцій між профілями (перемикання профілю змінює вміст «Бібліотеки» та «Мого списку»), виключення контенту для дорослих із загальнодоступних добірок та коректне відображення градієнтного замітника за відсутності обкладинки.

3.3 Перевірка відмовостійкості та обходу обмежень

Підтверджено усунення проблеми порожніх рядів: після впровадження обмеження темпу звернень добірки, що формуються одночасно, наповнюються повністю, оскільки звернення більше не відхиляються джерелом за перевищення темпу. Перевірено градаційне зниження функціональності: за недоступності зовнішнього джерела підсистема повертає порожню добірку чи резервний набір замість аварійного завершення, а сторінка лишається працездатною. Окремо підтверджено сценарій підміни джерела: після припи-

нення доступу до комерційного сервісу коміксів джерело замінено відкритим без зміни клієнтської частини, що засвідчує коректність відв'язування через комунікаційні інтерфейси та практичну цінність цього архітектурного рішення.

3.4 Показники продуктивності

Повторні звернення до каталогу обслуговуються з кешу за час порядку мілісекунд, без звернень до мережі. Статичний набір даних персонажів (близько 1 МБ) завантажується одноразово й кешується на тривалий інтервал, після чого пошук, фільтрація та посторінкове гортання виконуються у пам'яті; завдяки цьому перемикання фільтра видавців відбувається миттєво. Завдяки мінімальному інтервалу між зверненнями до Jikan дотримується обмеження темпу джерела навіть під час інтенсивного гортання каталогу.

3.5 Контроль якості та командна взаємодія

Розроблення велося із застосуванням системи контролю версій Git за моделлю гілок, запитів на злиття та взаємного рецензування коду. Типобезпеку клієнтської частини перевіряв компілятор TypeScript, складання — засоби Vite; стиль коду підтримувався статичним аналізатором. Інтеграцію з підсистемою облікових записів здійснено через узгоджені комунікаційні інтерфейси та спільний контекст автентифікації, що знизило зв'язність складових і спростило паралельну роботу учасників команди.

ВИСНОВКИ

У результаті виконання дипломної роботи спроектовано, реалізовано та випробувано контентну підсистему інформаційної системи розповсюдження мультимедійного контенту — агрегатор метаданих, уніфікований каталог, наскрізний пошук, персональні колекції та потокове відео. Поставлені у вступі задачі виконано повністю; нижче наведено результати в порядку їх формулювання.

1. Проведено порівняльний аналіз провідних систем розповсюдження контенту (табл. 1.1) та доступних джерел метаданих (табл. 2.1) за програмними інтерфейсами доступу. Встановлено критерії вибору джерел і визначено предмет удосконалення — уніфіковане подання різноманітного контенту з незалежністю від конкретного джерела.
2. Синтезовано уніфіковану інформаційну модель контенту та спроектовано кешувальний агрегатор за стратегією Cache-Aside з диференційованими інтервалами актуальності й узагальненим методом кешованого гортання, що усунув дублювання коду між однотипними методами.
3. Реалізовано серверні комунікаційні інтерфейси (контролер метаданих із 18 кінцевими точками) та сервіси інтеграції з джерелами за принципом інверсії залежностей; синтезовано механізм обмеження темпу звернень, що усунув відхилення звернень за перевищення лімітів і пов'язану з ним проблему порожніх рядів.
4. Забезпечено підмінність постачальника метаданих через комунікаційний інтерфейс (підтверджено заміною припиненого комерційного сервісу відкритим джерелом без зміни клієнта); реалізовано уніфікований каталог зі спільним компонентом подання, наскрізний пошук за чотирма типами контенту та поточний перегляд відео вбудованим програвачем HTML5.
5. Реалізовано підсистему персональних колекцій («Бібліотека» та «Мій список») з прив'язкою до профілю; підтверджено, що кожен профіль має власні колекції.
6. Проведено функціональне випробування та перевірку відмовостійкості; підтверджено наповнення рядів після впровадження обмеження темпу звернень, обслуговування повторних звернень з кешу за час порядку мілісекунд та коректність підміни джерела без зміни клієнта.

Новизна роботи полягає не у відтворенні відомого ресурсу, а у створенні вдосконаленого варіанта: синтезовано уніфіковану інформаційну модель подання різноманітного контенту, застосовано кешування з обмеженням темпу звернень та відв'язування від постачальника метаданих. Практична цінність полягає в тому, що обґрунтований шаблон агрегатора метаданих та результати порівняльного аналізу можуть бути повторно використані наступними розробниками.

Рекомендації та перспективи. Доцільним є впровадження адаптивного потокового відео (HLS/DASH), застосування мережі доставки контенту (CDN) для статичних ресурсів та побудова рекомендаційної підсистеми на основі колаборативної фільтрації. Загальний висновок проєкту формується як сукупність звітів усіх учасників команди.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. ASP.NET Core documentation [Електронний ресурс] / Microsoft. URL: <https://learn.microsoft.com/aspnet/core/>
2. Entity Framework Core documentation [Електронний ресурс] / Microsoft. URL: <https://learn.microsoft.com/ef/core/>
3. Caching in .NET: IMemoryCache [Електронний ресурс] / Microsoft. URL: <https://learn.microsoft.com/aspnet/core/performance/caching/memory>
4. RFC 7231. HTTP/1.1: Semantics and Content [Електронний ресурс] / R. Fielding, J. Reschke. IETF, 2014. URL: <https://www.rfc-editor.org/rfc/rfc7231>
5. IHttpConnectionFactory in .NET [Електронний ресурс] / Microsoft. URL: <https://learn.microsoft.com/dotnet/core/extensions/httpclient-factory>
6. Cache-Aside pattern [Електронний ресурс] / Microsoft Azure Architecture Center. URL: <https://learn.microsoft.com/azure/architecture/patterns/cache-aside>
7. MySQL 8.0 Reference Manual [Електронний ресурс] / Oracle. URL: <https://dev.mysql.com/doc/refman/8.0/en/>
8. HTML Living Standard: The video element [Електронний ресурс] / WHATWG. URL: <https://html.spec.whatwg.org/multipage/media.html>
9. React documentation [Електронний ресурс] / Meta Open Source. URL: <https://react.dev/learn>
10. TypeScript documentation [Електронний ресурс] / Microsoft. URL: <https://www.typescriptlang.org/docs/>
11. Vite documentation [Електронний ресурс]. URL: <https://vite.dev/guide/>
12. TanStack Query documentation [Електронний ресурс]. URL: <https://tanstack.com/query/latest>
13. Jikan REST API (v4) documentation [Електронний ресурс]. URL: <https://docs.api.jikan.moe/>
14. TVMaze API documentation [Електронний ресурс]. URL: <https://www.tvmaze.com/api>
15. Open Library Developer APIs [Електронний ресурс] / Internet Archive. URL: <https://openlibrary.org/developers/api>
16. akabab/superhero-api [Електронний ресурс]. URL: <https://akabab.github.io/superhero-api/>
17. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. Київ: ДП «УкрНДНЦ», 2016. 26 с.
18. ДСТУ 8302:2015. Інформація та документація. Бібліографічне посилання. Загальні положення та правила складання. Київ: ДП «УкрНДНЦ», 2016. 17 с.

ДОДАТОК А. ТЕХНІЧНЕ ЗАВДАННЯ ПРОЄКТУ

Загальний опис

Програмне забезпечення реалізує розподілену інформаційну систему розповсюдження мультимедійного контенту за моделлю підписки (робоча назва — Bazinga). Цей звіт присвячено контентній підсистемі — агрегатору метаданих, уніфікованому каталогу, наскрізному пошуку, потоковому відео та персональним колекціям, яка є спільною основою двох предметних складових системи (Bazinga Comics та BazingaTV).

Призначення

Програмне забезпечення призначене для надання користувачеві можливості переглядати каталог різноманітного контенту з обкладинками, описами й жанрами; здійснювати наскрізний пошук; зберігати одиниці контенту до персональних колекцій; переглядати відеоконтент потоково. Підсистема забезпечує уніфіковане подання контенту з різних джерел та збереження працездатності за відмови окремих зовнішніх джерел.

Цілі та задачі системи

Система повинна забезпечувати: агрегування метаданих з кількох зовнішніх джерел через єдину точку доступу; уніфіковане подання різноманітного контенту; фільтрацію, сортування та посторінкове гортання; наскрізний пошук; персональні колекції на рівні профілю; потоковий перегляд відео; стійкість до відмови й обмежень зовнішніх джерел.

Функціональні вимоги

1. Каталог і метадані. Система повинна забезпечувати: єдину точку доступу до метаданих з кількох зовнішніх джерел; уніфіковане подання коміксів, манги, персонажів та серіалів однаковими інтерфейсними елементами; фільтрацію за жанрами та видавцями; посторінкове гортання; виключення контенту для дорослих із загальнодоступних добірок.

2. Пошук. Система повинна забезпечувати: наскрізний пошук одночасно за коміксами, мангою, аніме та персонажами; відкриття детального подання результату безпосередньо в накладенні без переходу на іншу сторінку; усунення зайвих звернень під час введення (debounce).

3. Персональні колекції. Користувач повинен мати можливість: зберігати одиниці контенту до колекцій «Бібліотека» (комікси й манга) та «Мій список» (аніме й серіали); розподіляти колекцію за типами контенту; мати окремі колекції для кожного профілю облікового запису.

4. Потокowe відео. Система повинна забезпечувати: головну сторінку BazingaTV з добірками контенту та героїчним блоком із трейлерами; повноекранний перегляд у вбудованому програвачі стандарту HTML5.

Нефункціональні вимоги

1. Продуктивність. Мінімізація кількості звернень до зовнішніх джерел кешуванням; обслуговування повторних звернень з кешу за час порядку мілісекунд; дотримання обмежень темпу звернень зовнішніх джерел.

2. Масштабованість. Незалежність контролера від конкретного джерела даних; можливість підключення нового джерела через інверсію залежностей без зміни наявного коду.

3. Надійність та відмовостійкість. Збереження працездатності за відмови зовнішнього джерела (порожня добірка чи резервний набір замість аварійного завершення); підмінність постачальника метаданих без зміни клієнтської частини.

4. Адаптивність. Коректна робота у браузерях Google Chrome, Mozilla Firefox, Safari, Microsoft Edge та у мобільних браузерах; адаптація інтерфейсу під десктопи, планшети та смартфони.

Тестування

Передбачено функціональне, інтеграційне, навантажувальне, кросбраузерне та адаптивне тестування; у межах контентної підсистеми основну увагу приділено функціональному випробуванню кінцевих точок і перевірці стратегій кешування та відмовостійкості.

Критерії готовності

Проект вважається виконаним, якщо: реалізовано всі функціональні вимоги; відсутні критичні помилки; підсистема коректно працює на всіх погоджених пристроях і браузерах; виконано вимоги щодо відмовостійкості; успішно проведено тестування; складено звітну частину.